ATOMIC SQUIRREL

THE STARTUP CHECKLIST

- [ ] **Idea**
  - [ ] **Problem**
    See Startup Ideas.
  - [ ] **Validation**
    See The Four Steps to the Epiphany.
  - [ ] **Market sizing**
    See Market Sizing Resources.
  - [ ] **Viability**
    Can this problem be solved? Can this problem be solved by *you*? See Sequoia's Why Now? and The Startup of You.
  - [ ] **Alternatives**
    See Letter To A Young Programmer Considering A Startup.
  - [ ] **Education**
    How do you build a startup? How do you design products? How do you build the technology? How do you hire people? See Hello, Startup, Hello, Startup Reading List, and How to Start a Startup.
  - [ ] **Friends and family**
    A startup is a big commitment. Make sure everyone is on board before you get started.
- [ ] **Legal**
  - [ ] **Lawyer**
    See What Start-ups Should Know About Hiring a Lawyer.
  - [ ] **Company name**
    See How to Name a Business.
  - [ ] **Incorporation**
    See When You Need to Incorporate Your Startup and The Entrepreneur's Guide to Business Law.
  - [ ] **Equity**
    See Startup Equity Resources.
  - [ ] **Licenses and permits**
    See What State Licenses and Permits Does Your Business Need?
  - [ ] **Federal and state Employer ID Numbers**
    See Employer ID Numbers.
  - [ ] **Insurance**
    See What Type of Insurance Do I Need? and 13 Types of Insurance a Small Business Owner Should Have.
  - [ ] **Intellectual Property**
    See Intellectual property for startups and The Entrepreneur's Guide to Business Law.
  - [ ] **Contracts**
    Work with your lawyer to draft contracts for your customers, employees, contractors, partners, suppliers, and so on.
- [ ] **Financial**
  - [ ] **Accountant**
    See Why Startups Need Accountants and 5 Reasons Why Your Start-Up Needs an Accountant.
  - [ ] **Accounting system**

See Accounting 101 for tech startups, Small Business Accounting 101, and How to Start a Startup Lecture 18: Legal and Accounting Basics for Startups.

- [ ] **Taxes and annual reports**

  Every year, you must pay taxes and file annual reports to state and federal governments. These are due at different times throughout the year, so make sure to figure out the deadlines ahead of time and get them plugged into your calendar.

- [ ] **Business bank account and credit card**

  See Why, and How to Keep Your Business and Personal Banking Separate, Why You Need A Business Banking Account, And How To Get One, Business Checking Accounts, and Business Credit Cards.

- [ ] **Monetization**

  See Developing a Business Plan and Startup Monetization Resources.

- [ ] **Pitch deck**

  See The Art of the Pitch and Pitch Anything.

- [ ] **Raise money**

  See How to Startup a Startup Lecture 9: How to Raise Money, How to Raise Money, and A Fundraising Survival Guide.

  - [ ] **Bootstrap**

    Fund a startup with your own money and by generating revenue early on.

  - [ ] **Loans**

    Fund a startup by borrowing money in exchange for paying it back later, with interest.

  - [ ] **Equity financing**

    Fund a startup by raising money from investors in exchange for part ownership of the company. See Venture Deals and Mastering the VC Game.

  - [ ] **Crowdfunding**

    Fund a startup by raising money from a large number of people online. There are three primary types of crowdfunding: rewards-based, where you offer rewards or the actual product in exchange for the money (e.g., KickStarter, IndieGoGo); debt-based, where you eventually pay the money back, similar to a loan (e.g., Lending Club, Prosper); and equity-based, where you give investors part-ownership of your company in exchange for the money (e.g., Crowdfunder, Wefunder).

- [ ] **Product**

  - [ ] **Design**

    See Design Resources.

    - [ ] **Platform**

      What platforms will you need to support? Desktop web? Mobile web? Native mobile apps (iOS, Android, etc.)? IoT? Which form factors? Desktop? Phone? Tablet? Wearable? And what design philosophy will you use? Mobile first? Web first? Phone first?

    - [ ] **Personas and user stories**

      See Personas, DIY User Personas, User Stories, and The Inmates Are Running the Asylum.

    - [ ] **Design reuse**

      Good software developers reuse as much battle-tested code as possible. Good designers reuse as many battle-tested designs as possible. See Design Resources and Photos and Graphics Resources.

    - [ ] **Copywriting**

      Copywriting is more important to a design than colors, borders, pictures,and fancy animations. Don't put off copywriting to the end and don't replace it with lorem ipsum. See Getting Real.

    - [ ] **Visual design**

      See Design Resources.

    - [ ] **Usability testing**

      See Don't Make Me Think.

  - [ ] **Minimum Viable Product**

    See The Lean Startup, The Guide to Minimum Viable Product, and Minimum Viable Product Resources.

    - [ ] **MVP Type**

Landing page, explainer video, crowdfunding campaign, Wizard of Oz MVP, and Piecemeal MVP, etc. See The Ultimate Guide to Minimum Viable Products.

- [ ] **Focus on the differentiators**

  If your product is great, it doesn't need to be good.

- [ ] **Target earlyvangelists**

  See Earlyvangelists and The Four Steps to the Epiphany.

- [ ] **MVP tools and resources**

  See Minimum Viable Product Resources.

- [ ] **Data**

  It is the product manager's job to define (1) What game are we playing? and (2) How do we keep score?. The best way to keep score is use data.

  - [ ] **AARRR Metrics**

    Every startup needs to track at least 5 key metrics: Acquisition, Activation, Retention, Referral, and Revenue (easy to remember by the acronymn AARRR from Dave McClure's Start Metrics for Pirates).

  - [ ] **The magic number**

    See Discovering your aha moment and How do you find insights like Facebook's "7 friends in 10 days" to grow your product faster?

  - [ ] **Data-driven development**

    See Data Driven Products Now!

- [ ] **Distribution**

  The best product doesn't always win. It's the best product the customer knows about that wins. See Startup Distribution Resources.

  - [ ] **Marketing**

    Word of mouth, advertising, public relations (PR), search engine optimization (SEO), social media, inbound marketing. See Startup Distribution Resources.

  - [ ] **Sales**

    Automated sales, inbound sales, outbound sales. See Startup Distribution Resources.

  - [ ] **Branding**

    Branding isn't about a single ad campaign or a logo or any one thing, but the sum total of all the ways you interact with the customer and how they think about your business. It's not about features or specs, but about the promise you're making to your customers. See Features vs Benefits and Steve Jobs on "Think Different" for more information.

- [ ] **Technology**

  - [ ] **Server-side**

    - [ ] **Programming language**

      Go with what you know.

    - [ ] **Server-side framework**

      See Node.js vs Play Framework for an example of how to compare frameworks.

    - [ ] **Architecture**

      See The Twelve-Factor App, Microservices, and stream processing, event sourcing, and CQRS.

    - [ ] **Scalability & Performance**

      See Scalability Resources.

  - [ ] **Client-side**

    What clients do you support? Desktop web? Mobile web? Native mobile apps (iOS, Android, etc.)? Wearables? IoT?

    - [ ] **Native apps, cross-platform app, or web-app**

      Write multiple apps in multiple native languages (Objective C or Swift, Java, C#, etc.), or write all in one common language that is transcompiled to native languages (PhoneGap, Titanium, Parse, etc.), or write only for the web (HTML5).

    - [ ] **Client-side framework**

      See TodoMVC, List of JavaScript Libraries, microjs, and CSS Preprocessors.

    - [ ] **SEO**

See Google Search Engine Optimization Starter Guide, Moz Beginner's Guide to SEO, and Search Engine Land's Guide To SEO.

- [ ] **Accessibility**

  See Introduction to Web Accessibility, The Accessibility Project, a11y, and ARIA.

- [ ] **Meta tags**

  See How to Use HTML Meta Tags, Facebook Open Graph Markup, Twitter Cards, and Favicon Generator.

- [ ] **Terms & conditions and privacy policy**

  See TermsFeed, Website Terms and Conditions Template, and Privacy Policy Template.

- [ ] **Developer accounts**

  See Apple Developer Program, Google Play Developer, Windows App Developer, and BlackBerry Developer.

- [ ] **Browser performance tools**

  YSlow, PageSpeed

- [ ] **CDN**

  See CloudFlare Free CDN, Amazon CloudFront, and Akamai CDN.

- [ ] **HTTP Caching**

  See HTTP caching.

- [ ] **Compression**

  See Optimizing encoding and transfer size of text-based assets.

- [ ] **Static content**

  Minify, concatentate, put stylesheets at the top, scripts at the bottom, pre-compile, use versioning or fingerprinting.

- [ ] **Lazy load**

  See async vs defer attributes and lazySizes.

- [ ] **BigPipe**

  See BigPipe: Pipelining web pages for high performance and Ping-Play: Big Pipe Streaming for the Play Framework.

- [ ] **HTML5 preconnect, prefetch, prerender**

  See Preconnect, prefetch, prerender.

- [ ] **Data storage**

  See Designing Data-Intensive Applications and NoSQL Distilled.

  - [ ] **Schema management**

    Manage schemas in dev, test, and production, enforce integrity constraints, and evolve the schemas. Remember, schemaless databases don't exist.

  - [ ] **Data processing**

    See OLAP, Hadoop, Kafka, Samza, and Spark.

  - [ ] **Query tuning**

    Use EXPLAIN, the query optimizer, indices, hints, precomputed views, denormalized data.

  - [ ] **Database tuning**

    Tweak memory settings, CPU settings, I/O settings.

  - [ ] **Caching**

    Tune database cache settings or use a distributed cache in front of the database, such as memcached.

  - [ ] **Replication**

    See Replication.

  - [ ] **Partitioning**

    See Partioning.

- [ ] **Continuous Integration**

  See Continuous integration.

  - [ ] **Version Control System (VCS)**

    See Why should I use version control?, Git, SVN, Git vs SVN, and trunk-based development.

  - [ ] **Build system**

See List of build automation software.

- [ ] **Dependency management**

  See Explicitly declare and isolate dependencies and Java Build Tools: How Dependency Management Works with Maven, Gradle and Ant + Ivy.

- [ ] **Automated tests**

  See Test Automation and Growing Object-Oriented Software, Guided by Tests.

- [ ] **Automated build servers**

  See Travis CI, Circle CI, and Jenkins.

- [ ] **Static analysis**

  See static analysis and code coverage.

- [ ] **Continuous Delivery**

  See Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.

  - [ ] **Deployment environments**

    See Deployment Environment.

  - [ ] **Hosting**

    See Startup Hosting Resources.

  - [ ] **Configuration management**

    See Deployment Tools.

  - [ ] **Automated deployment**

    See Deployment Tools.

  - [ ] **Canary releases**

    See Canary Release.

  - [ ] **Feature toggles and A/B testing**

    See Feature Toggles, Using Feature Flags to Ship Changes with Confidence, PlanOut, Optimizely, Vanity, Amazon A/B testing.

  - [ ] **Change management dashboard**

    Create a dashboard that tracks all the changes happening to production services, including new deployments, configuration changes, hardware upgrades, feature flag changes, and so on. When something goes wrong, this dashboard will be the first place you turn to answer the question, "What changed?"

- [ ] **Monitoring**

  See Monitoring Resources.

- [ ] **Security**

  - [ ] **Network isolation**

    Control which IPs and ports are accessible on all servers. Segregate different parts of your stack into separate networks: e.g., one for your corporate intranet, one for your backend servers, one for your public-facing web servers. See Amazon VPC.

  - [ ] **SSL**

    See How To Install an SSL Certificate from a Commercial Certificate Authority, How to obtain and install an SSL/TLS certificate, for free, and Beginner's Guide To Website SSL Certs.

  - [ ] **Secret storage**

    See Vault, Keywhiz, Sneaker, Lemur, and BlackBox.

  - [ ] **User authentication**

    See Authentication Cheat Sheet, Password Storage Cheat Sheet, Session Management Cheat Sheet, Best way to store password in database, and Auth0.

  - [ ] **Server hardening**

    See Hardening the Linux server, 25 Hardening Security Tips for Linux Servers, 20 Linux Server Hardening Security Tips, 13 Apache Web Server Security and Hardening Tips, and Server security best practices.

  - [ ] **CSRF**

    See CSRF Prevention Cheat Sheet.

  - [ ] **XSS**

    See XSS Prevention Cheat Sheet.

- [ ] **Injection attacks**

  See Code Injection, SQL Injection Prevention Cheat Sheet, Eval Injection.

- [ ] **Security headers**

  See List of useful HTTP Headers.

- [ ] **Security audit**

  See Information technology security audit.

- [ ] **Security advisories**

  For every commercial and open source piece of software your rely on, including your operating system, programming language, web framework, database, and client-side libraries, make sure you can sign up for a security advisory mailing list that will notify you when vulnerability is discovered. Make sure you have the ability to update to a newer version of the software quickly, before hackers can exploit it.

- [ ] **Spam and fraud**

  Does your product have a lot of user-generated content? If so, you may need a way to prevent fraud and spam. At the least, you need something like CAPTCHA to protect against automated attacks. To protect against manual attacks, you may have to hire a fraud team.

- [ ] **Clean code**

  - [ ] **Coding standards**

    Define coding conventions for your project, including a style guide, testing requirements, and documentation requirements.

  - [ ] **Code ownership**

    Identify an owner for each part of the code who is responsible for deploying the code, keeping it working, and deciding what changes can go in.

  - [ ] **Code reviews**

    See Code Review Resources.

  - [ ] **Documentation**

    See You Are What You Document.

  - [ ] **Split up the code**

    It can be hard to reason and make changes to a single, giant, monolithic codebase, especially when many people are making changes to it at the same time. Two possible, orthogonal solutions: one is to break the code up into separate repositories. Another option is to break the code into separate services (AKA microservices) that can be developed and deployed completely in isolation.

- [ ] **Common functionality**

  A partial list of common features that almost every startup has to build.

  - [ ] **Search**

    See Solr, Elasticsearch, Algolia, Searchify, and Amazon CloudSearch.

  - [ ] **Email**

    See SendGrid, Mailgun, Amazon SES, and Mandrill.

  - [ ] **Notifications**

    See Amazon SNS, Parse Push, Urban Airship, and Pushover.

  - [ ] **User Management**

    See Auth0, Janrain, Stormpath, and OneAll.

  - [ ] **News feed**

    See Stream and Stream-Framework.

  - [ ] **Monetization**

    See Startup Monetization Resources.

  - [ ] **Microservice management**

    See Kong, Amazon API Gateway, Vamp, SmartStack, Eureka, and Serf.

  - [ ] **File storage and upload**

    See Amazon S3, RackSpace Cloud Files, and Google Cloud Storage.

- [ ] **Team**

- [ ] **Culture**

  See From Vision to Values: The Importance of Defining Your Core, Culture trumps strategy, every time, How to Start a Startup Lecture 10: Culture, How to Start a Startup Lecture 11: Hiring and Culture, Part II, and Don't Fuck up the Culture.

  - [ ] **Mission and values**

    See Netflix Culture Deck, HubSpot Culture Code, How Great Leaders Inspire Action.

  - [ ] **Organizational design**

    See Management, The Hard Thing About Hard Things, Valve Handbook for New Employees, Holocracy, Thoughts on the Technical Track, and Organizational Charts.

  - [ ] **Process**

    Small startups don't need much process use good judgment is usually enough. Only when good judgment has been repeatedly shown not to work should you consider adding any sort of process, and even then, do so carefully, revisit often, and prefer automation to manual processes (see The Agile Manifesto).

  - [ ] **Communication**

    How will you communicate with people within the company? How will you communicate with people outside? How transparent will you be? What information will you reveal? For a radical example, check out the startup Buffer.

  - [ ] **Motivation**

    See The puzzle of motivation, Drive: The Surprising Truth About What Motivates Us, Can Money Buy You Happiness?, 5 Tips on How to Give Employees Purpose, LinkedIn HackDays, Google 20% time, and Valve 100% time.

- [ ] **Office**

  - [ ] **Type of office**

    Will you lease an office? Join a co-working space? Work out of your living room or garage? Or will you build a distributed company where there is no office, and everyone works from home?

  - [ ] **Location**

    What kind of commute will you and your employees have? Is there parking available? Are there places to eat nearby? Is there room for future growth? What are the zoning laws for this type of business?

  - [ ] **Electrical**

    Power, back-up power, lighting, Internet (Wi-Fi and ethernet), phone.

  - [ ] **Temperature control**

    Heating, air conditioning, windows, shades.

  - [ ] **Bathroom facilities**

    Enough for the number of employees.

  - [ ] **Kitchen facilities**

    At the very least, you need easy access to water, coffee, and basic snacks.

  - [ ] **Access control**

    Keys and/or swipe cards at every exterior door. Consider if you need to hire security guards.

  - [ ] **Fire safety measures**

    Fire extinguishers, smoke detectors, sprinklers, emergency exits.

  - [ ] **Space for focused work**

    Offices, cubicals, open floor plans. See The Open-Office Trap and Why You Shouldn't Interrupt a Programmer.

  - [ ] **Space for collaborative work**

    Conference rooms and common areas.

  - [ ] **Furniture**

    Desks, chairs, cubicles, filing cabinets, lamps, conference tables, white boards, TVs, projectors, couches.

  - [ ] **Supplies**

    Pens, pencils, notepads, sticky notes, tape, printer paper, folders, markers, erasers, envelopes, stamps.

  - [ ] **IT**

Computers, monitors, keyboards, mice, printers, copiers, fax machines, phones, software.

- [ ] **Communication**

  See Startup Collaboration Resources for more info.

  - [ ] **Email**

    See Google Apps for Business.

  - [ ] **Chat**

    See Slack, Campfire, and Yammer.

  - [ ] **Phone & fax**

    See Grasshopper, Google Voice, and Talkroute.

  - [ ] **Wiki**

    See MediaWiki, Atlassian Confluence, and SlimWiki.

  - [ ] **Project management & issue tracker**

    see Trello and Asana.

  - [ ] **Business cards**

    See Vistaprint.

- [ ] **Hiring**

  See A Guide to Hiring for your Startup

  - [ ] **Co-founders**

    You need a co-founder to handle part of the workload, fill in your weaknesses, and to talk you out of bad ideas. Startups are hard and without the motivation of "I can't let my cofounder down", you're not likely to succeed. A team of 2-3 co-founders works best. For more information, see A Guide to Hiring for your Startup.

  - [ ] **Advisors**

    Everyone needs a mentor, coach, or advisor. All companies that are C Corporations (i.e., almost all venture-backed startups) have a board of directors. Find people who you can trust, who fill in the gaps in your skills, and who have a network you can use.

  - [ ] **Referrals**

    See Why Employee Referrals are the Best Source of Hire and Recruit Engineers in Less Time.

  - [ ] **Employer branding**

    See CEOs Need to Pay Attention to Employer Branding and 15 Employer Branding Best Practices You Need To Know.

  - [ ] **Job boards**

    See Your Job Posting Sucks and Startup Job Resources.

  - [ ] **The interview**

    See A Guide to Hiring for your Startup.

  - [ ] **The offer**

    See Startup Salary Resources and Startup Equity Resources.

  - [ ] **Promotions**

    See Thoughts on the Technical Track and The Peter Principle.

Share:

Credits: Rocket photograph by manhhai; logo design by Pirog tetyana; website design by HTML5 Up.

Copyright: Atomic Squirrel, LLC 2016. All rights reserved.